

Trabajo Práctico N° 3: Arquitectura, Escalabilidad y Gestores NoSQL

UNIVERSIDAD TECNOLÓGICA NACIONAL (UTN)

Materia: Bases de Datos I

Alumno: Patricio Sussini Guanziroli

ACTIVIDAD 1: Solución de Infraestructura (Caso: "Ticket-All")

1. Escalabilidad Vertical vs. Horizontal

La propuesta de la empresa de triplicar la memoria RAM se enmarca en lo que conocemos como **Escalabilidad Vertical** (Scaling Up). Si bien es la solución más sencilla de implementar inicialmente, tiene un "techo" técnico y económico infranqueable: el hardware tiene límites físicos. Por más RAM que se añada, el procesador o el ancho de banda del bus de datos eventualmente se convertirán en un cuello de botella. Además, esta arquitectura genera un **punto único de falla**; si ese servidor falla durante el pico de ventas, todo el sistema queda fuera de servicio.

En mi opinión, para un caso de alta demanda como la venta de entradas de artistas internacionales, la **Escalabilidad Horizontal** (Scaling Out) es la opción correcta. Esta permite distribuir la carga entre múltiples servidores económicos. A diferencia de la vertical, no tiene un techo real, ya que siempre podemos seguir añadiendo nodos al cluster según la demanda lo requiera.

2. Arquitectura de Sharding

Para evitar que el sistema colapse, propongo una arquitectura de **Sharding**.

- **Shard Key elegida:** Seleccionaría el User_ID aplicando una función de Hash.
- **Justificación:** El uso de un Hash sobre el ID de usuario garantiza que los compradores se distribuyan de manera uniforme entre todos los shards disponibles, independientemente de cuándo se conecten.
- **Punto Caliente (Hotspot):** Un "Hotspot" ocurre cuando una gran cantidad de operaciones se dirigen a un solo shard mientras los demás están ociosos (por ejemplo, si usáramos la fecha de compra como clave, todas las compras del momento irían al mismo servidor). Para evitarlo en este escenario, la clave elegida debe tener alta cardinalidad y una distribución aleatoria o no secuencial.

3. Replicación (Replica Sets)

La replicación es vital para la alta disponibilidad. En un Replica Set, tenemos los siguientes roles:

- **Nodo Primario:** Es el único que recibe todas las operaciones de escritura.
- **Nodos Secundarios:** Replican los datos del primario de forma constante.

Si el servidor primario falla en plena venta, el sistema no se detiene. Los nodos secundarios inician un proceso de "elección" automática para designar a un nuevo Primario. Este proceso es transparente para el usuario, asegurando que la venta de entradas continúe sin interrupciones.

ACTIVIDAD 2: Modelado y Consistencia (Caso: "Fintech-Plus")

1. ACID vs. BASE

Para los distintos flujos de datos de la aplicación, aplicaría los siguientes modelos:

- **Dato A (Salos y transferencias):** Aplicaría **ACID**. En transacciones financieras no podemos permitirnos ambigüedades. Se requiere **Consistencia Inmediata**; el usuario debe ver reflejado su saldo real al instante, garantizando que el dinero no exista en dos cuentas a la vez ni se pierda en el proceso.
- **Dato B (Muro de noticias y likes):** Aplicaría **BASE**. En este contexto, la disponibilidad es más importante que la precisión absoluta al milisegundo. Se utiliza **Consistencia Eventual**; si un "like" o un comentario tarda unos segundos en aparecer para todos los contactos, la integridad del sistema financiero no corre peligro y la experiencia del usuario se mantiene fluida.

2. Optimización mediante Desnormalización

En las bases de datos NoSQL, realizar JOINS entre colecciones es computacionalmente costoso y escala mal. Para mostrar el nombre y la foto del usuario en cada comentario de forma eficiente, utilizaría la **Desnormalización**.

Esto implica "incrustar" o duplicar los datos del usuario (nombre y URL de la foto) directamente dentro del documento del comentario. Aunque esto ocupa más espacio en disco, ganamos una velocidad de lectura inmejorable, ya que con una sola consulta obtenemos toda la información necesaria para renderizar el muro.

3. Flujo de datos en MongoDB Sharded Cluster

El camino de una consulta desde la App es el siguiente:

1. **App -> Mongos (Query Router):** El cliente se conecta al Mongos. Este componente actúa como un director de orquesta; recibe la consulta y, basándose en la Shard Key, sabe a qué servidor debe dirigirla.
2. **Config Servers:** El Mongos consulta a los Config Servers para obtener los metadatos.

Estos servidores guardan el mapa que indica qué rangos de datos están en qué shard.

3. **Shards:** Finalmente, la consulta llega al Shard correspondiente, se procesa y el resultado vuelve al usuario a través del router.

ACTIVIDAD 3: Investigación

1. Teorema CAP y Principio BASE

El **Teorema CAP** de Eric Brewer postula que en un sistema distribuido es imposible garantizar simultáneamente la Consistencia (C), la Disponibilidad (A) y la Tolerancia a Particiones (P). Dado que en internet las fallas de red (Particiones) son inevitables, los sistemas deben elegir entre ser Consistentes o estar Disponibles.

El **principio BASE** (Basically Available, Soft state, Eventual consistency) es la respuesta de NoSQL al teorema CAP: sacrifica la consistencia inmediata (ACID) para asegurar que el sistema siempre esté disponible y sea escalable.

- *Fuente: Documentación oficial de IBM y "NoSQL Distilled" (Pramod Sadalage).*

2. Redis vs. MongoDB para Leaderboards

Redis es notablemente más rápido que MongoDB para tablas de posiciones en tiempo real porque es una base de datos **In-Memory** (reside en RAM) y posee una estructura de datos específica llamada *Sorted Sets*. Mientras MongoDB debe buscar en disco e indexar, Redis opera directamente sobre estructuras matemáticas optimizadas en memoria.

Comandos de Redis:

- **ZADD**: Agrega un elemento con una puntuación a un conjunto ordenado (ideal para rankings).
- **INCR**: Incrementa de forma atómica el valor de una clave numérica (útil para contadores).
- **LPUSH**: Inserta un elemento al principio de una lista.
- *Fuente: Redis.io Documentation.*

3. Neo4j y la Detección de Fraude

Neo4j afirma que su modelo de **Grafos** es superior al de Documentos (MongoDB) para detectar fraude porque su estructura se basa en **nodos y aristas**.

En MongoDB, para encontrar conexiones entre personas, cuentas y direcciones, hay que realizar múltiples búsquedas o "lookups" complejos. En Neo4j, la relación es una entidad física con punteros directos. Esto permite realizar "recorridos de grafos" para detectar patrones como "anillos de fraude" (varias personas compartiendo el mismo teléfono o IP de forma sospechosa)

en milisegundos, algo que en un modelo de documentos requeriría procesos de unión extremadamente pesados.

- *Fuente: Neo4j Graph Data Platform - "Fraud Detection Use Case".*